ACM SIGARCH

1. BUREAUCRACY

What is SIGARCH?

- ACM x IEEE SIG dedicated to computer architecture
  - Revival of pre-existing SIG
- Introductory meetings on cool topics in computer architecture
  - Lower barrier to entry than course material
- Paper reading
  - Best way to understand modern computers? Read current research
- Projects?
  - No concrete plans yet but we'd like to gauge interest (form in Discord)

Who should join SIGARCH?

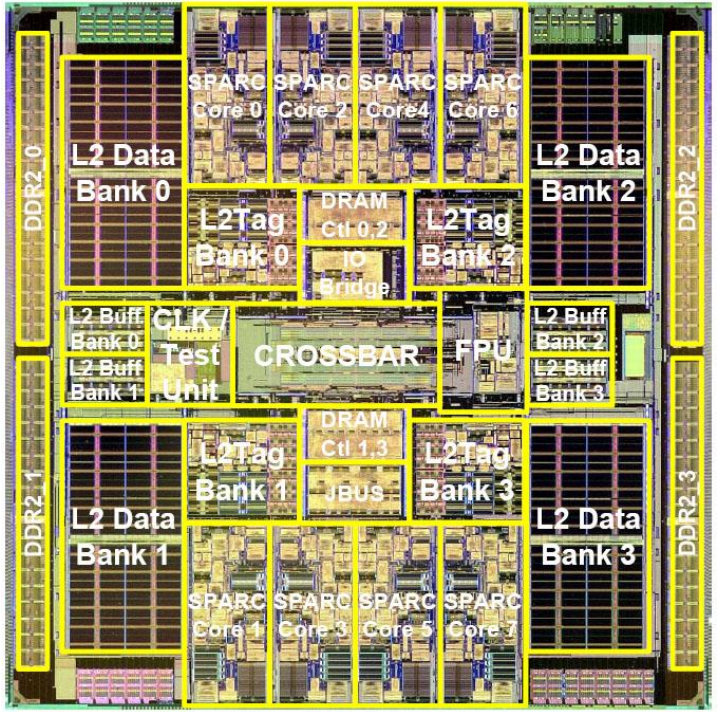
- Undergraduate and graduate students interested in computer architecture
  - If you're a grad student come talk to us we'd love your help
- Enjoyers of ECE 120, ECE 411, CS 233, and CS 433
  - We recommend having taken at least one of these
- Targeted at ECE and CS majors
  - Others are welcome too

2. INTRO TO COMPUTER ARCHITECTURE

Computers

(noun) an electronic device for storing and processing data, typically in binary form, according to instructions given to it in a variable program

95% of families in the US own a computer



Moore’s Law

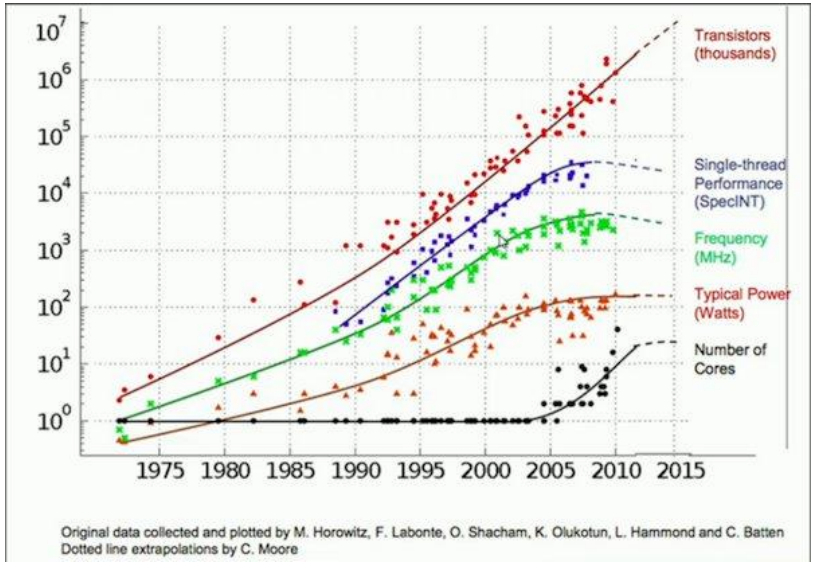
Moore’s Law says # of transistors doubles every 2 years (free speedup)

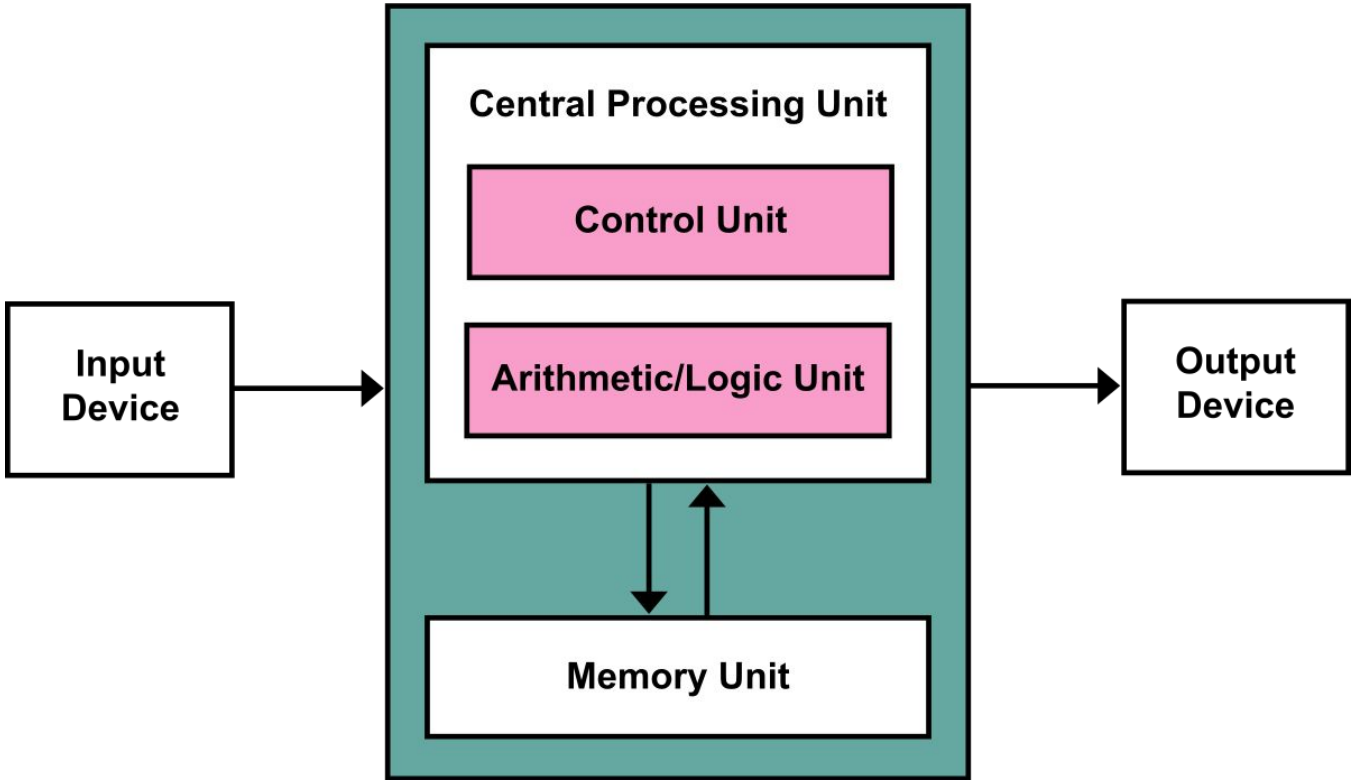
Moore’s Law is dead/dying

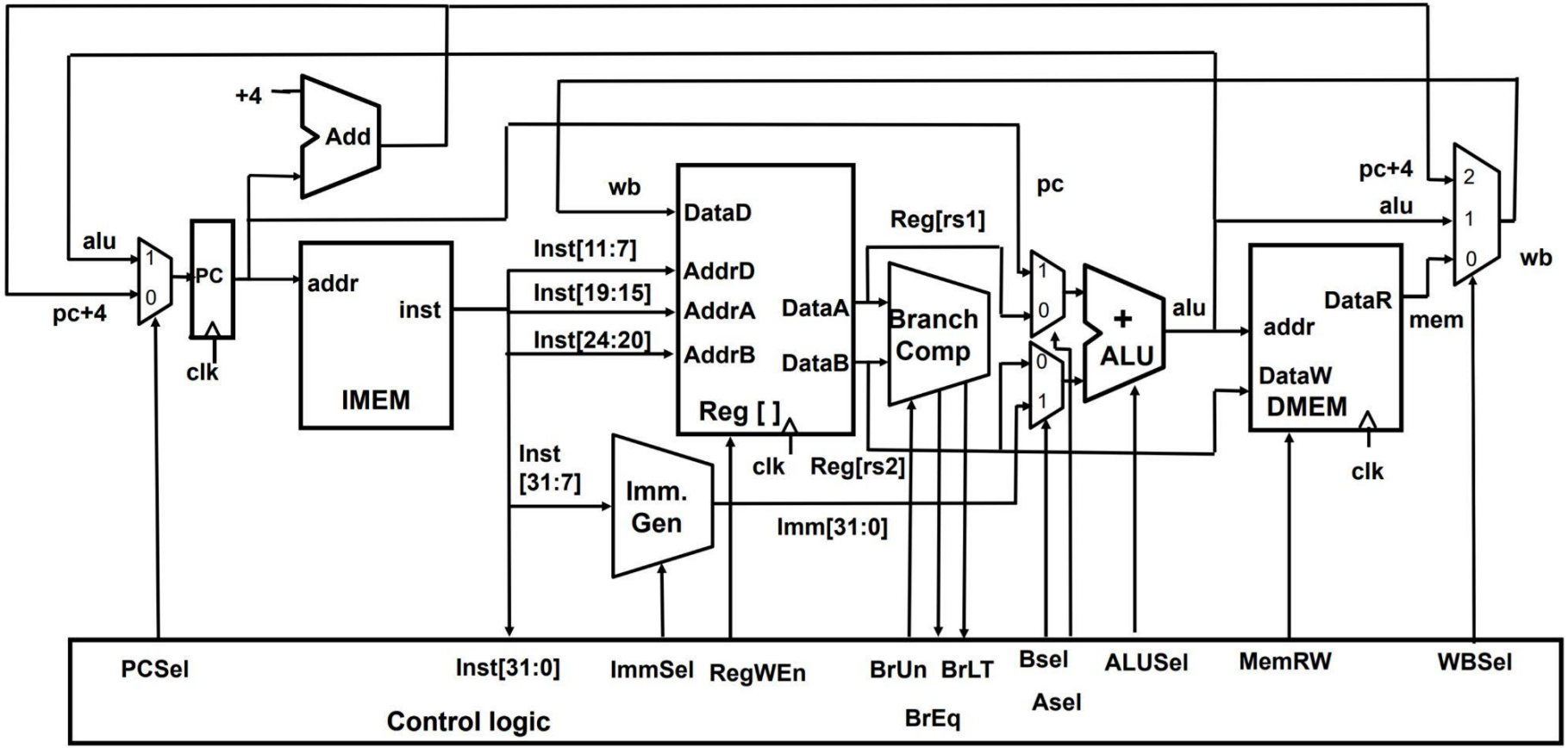
We still need faster computers ->

This means we need to be creative ->

Computer architecture research




Central Processing Unit
Control Unit
Arithmetic/Logic Unit
Input Device
Output Device
Memory Unit


+4
Add
alu
pc+4
1
0
PC
clk
addr
inst
IMEM
wb
DataD
AddrD
AddrA
AddrB
DataA
DataB
Reg [ ]
clk
Inst[11:7]
Inst[19:15]
Inst[24:20]
Inst
[31:7]
Imm.
Gen
Reg[rs1]
Reg[rs2]
Branch
Comp
pc
1
0
0
1
+
ALU
alu
Imm[31:0]
addr
DataR
DataW
DMEM
clk
mem
pc+4
alu
2
1
0
wb
PCSel
Inst[31:0]
ImmSel
RegWEn
BrUn
BrLT
BrEq
Bsel
Asel
ALUSel
MemRW
WBSel
Control logic

ISAs (Instruction Set Architecture)

Accumulator

+ Short instruction words
– High memory traffic, one primary on-chip register

Stack

+ Simple model of execution, short instruction words
– No random access

Register

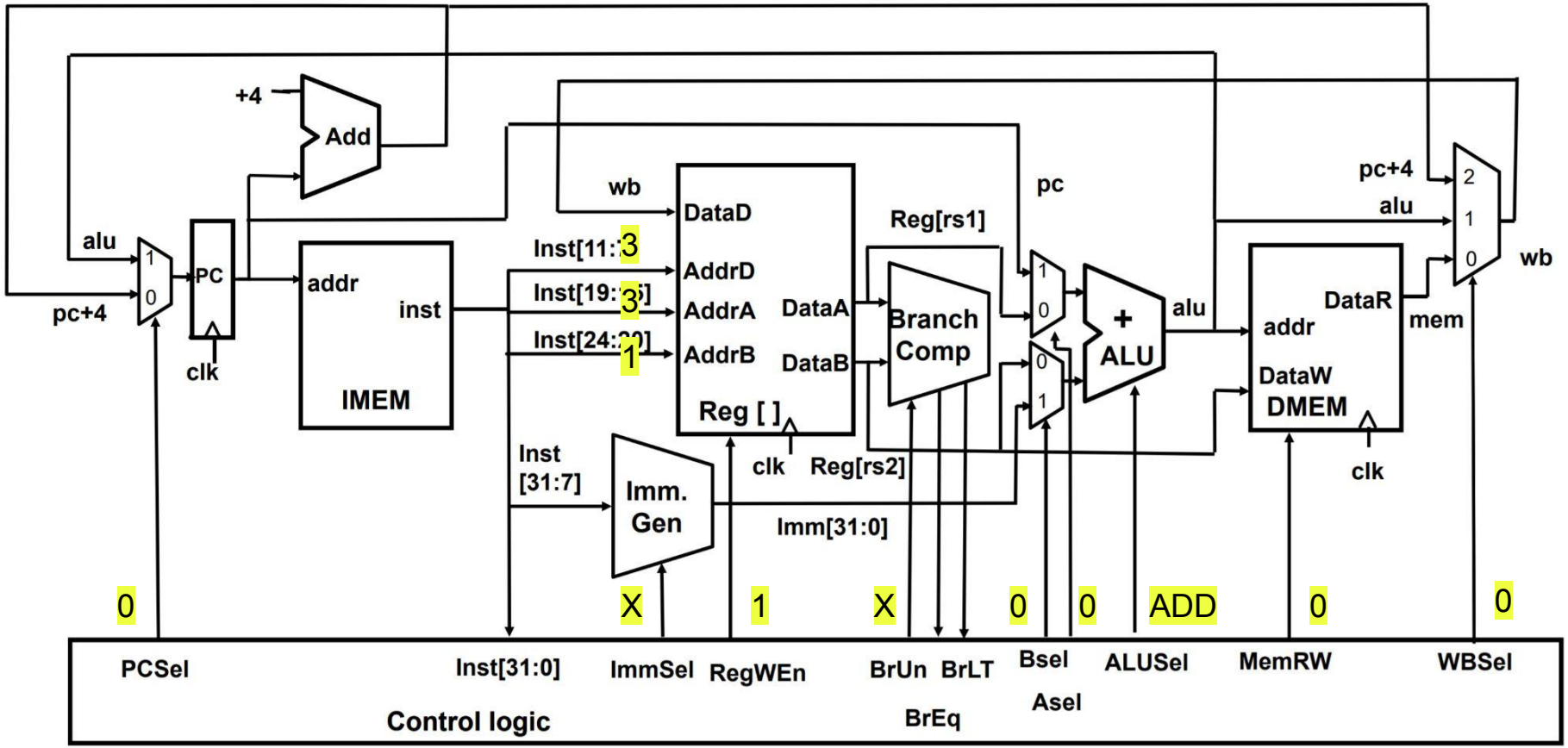
+ Easy code generation, on-chip storage
– Longer instructions (must specify operands)

RISC [Reduced-ISC] ➡ RISC-V, ARM

CISC [Complex-ISC] ➡ x86

+

ADD R3, R3, R1

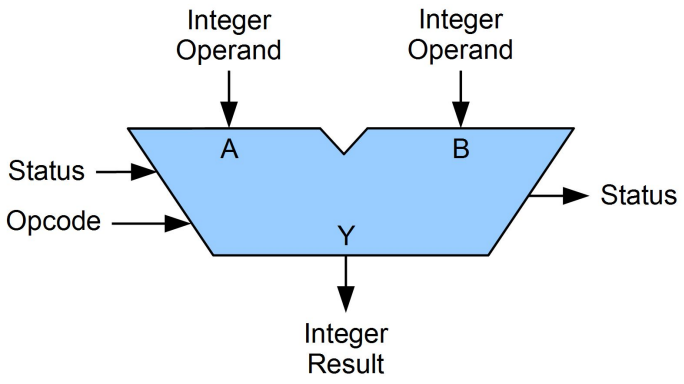

+4
Add
alu
pc+4
PC
clk
addr
inst
IMEM
wb
DataD
AddrD
AddrA
AddrB
DataA
DataB
Reg [ ]
clk
Inst[11:7]
Inst[19:15]
Inst[24:20]
Inst
[31:7]
Imm.
Gen
Reg[rs1]
Reg[rs2]
Branch
Comp
pc
+
ALU
alu
Imm[31:0]
addr
DataR
DataW
DMEM
clk
mem
pc+4
alu
wb
0
X
1
X
0
0
ADD
0
0
PCSel
Inst[31:0]
ImmSel
RegWEn
BrUn
BrLT
BrEq
Bsel
Asel
ALUSel
MemRW
WBSel
Control logic

Pipelining

- Notice that in the previous slide, the entire datapath is executing a single instruction.
- This isn’t necessary though, since when we’re executing an instruction, we can fetch and decode the next instruction, since the fetch/decode stage is not in use.
- And so on…
- This increases the *instruction throughput*, the latency of a single instruction is unaffected.
- Stalling.
- Most modern processors are pipelined.

Computers Can Do Math

- ALUs compute single-cycle logic and arithmetic functions
- MDUs (multiplication and division units)
  - Fast algorithms to multiply / divide
- x86 accelerates just about *everything*
  - Quake3 fast inverse sqrt is implemented in hardware!
  - SHA256 acceleration
  - AES (private key encryption algorithm) acceleration



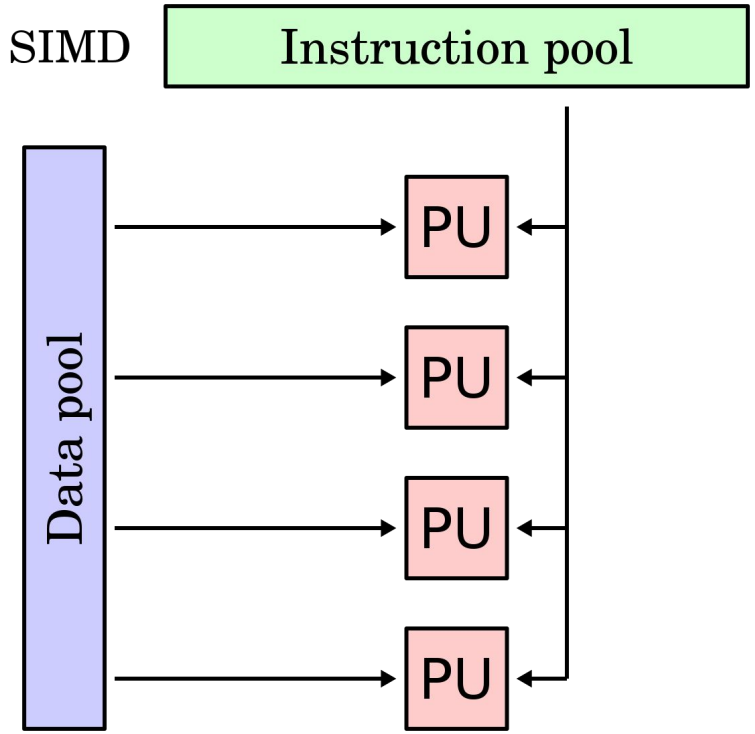

```
#include <bit>
#include <limits>
#include <cstdint>

constexpr float Q_rsqrt(float number) noexcept {
        static_assert(std::numeric_limits<float>::is_iec5
59); // (enable only on IEEE 754)

        float const y = std::bit_cast<float>(
                0x5f3759df -
(std::bit_cast<std::uint32_t>(number) >> 1));
        return y * (1.5f - (number * 0.5f * y * y));
}
```

SIMD

- Key insight: if you have a large register (say, 128 bits = 16 bytes).
- You can “interpret” these 128 FFs as:
  - Two 64-bit registers
  - Four 32-bit registers
  - Eight 16-bit registers
  - Sixteen 8-bit registers
- And interpret these as float or int depending on what you want to do.
- It’s easy to operate on 128 bits at a time in hardware, so we can efficiently compute the same operation on (say) eight 16-bit registers.
- Useful for multimedia applications/other places you want to do the same operation on multiple data points.



Memory

Computing is based on **data**, we need somewhere to store this data when it is not actively being used

DRAM is the most common type of memory

- Cheap
- Low Latency as compared to other storage options

Other types of memory may benefit systems, but all come with tradeoffs

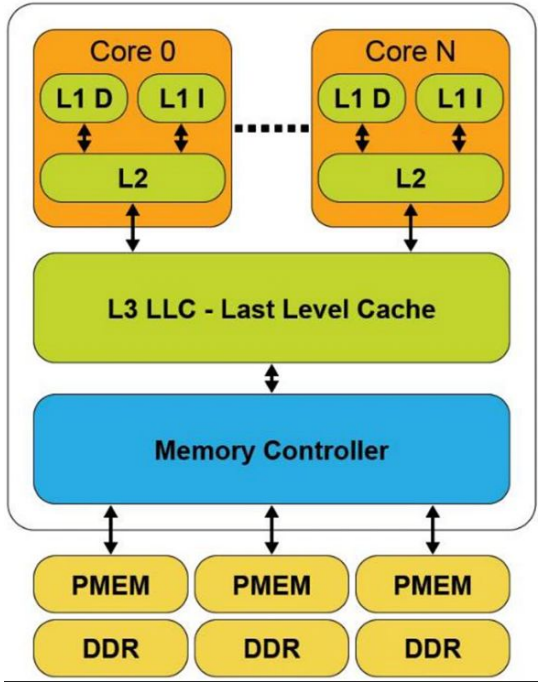
SRAM (faster, expensive), PCM (low power, higher latency), etc.

Caching

- Not enough registers to use as the primary near-device storage
- Memory access is high latency
  - How can we hide this latency?
- Caches are smaller data stores that keep recent data closer to processor

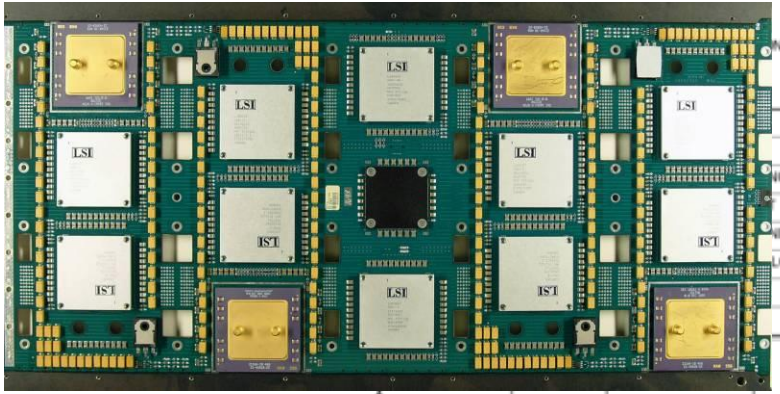
Indexing: VIVT, PIPT, VIPT, PIVT

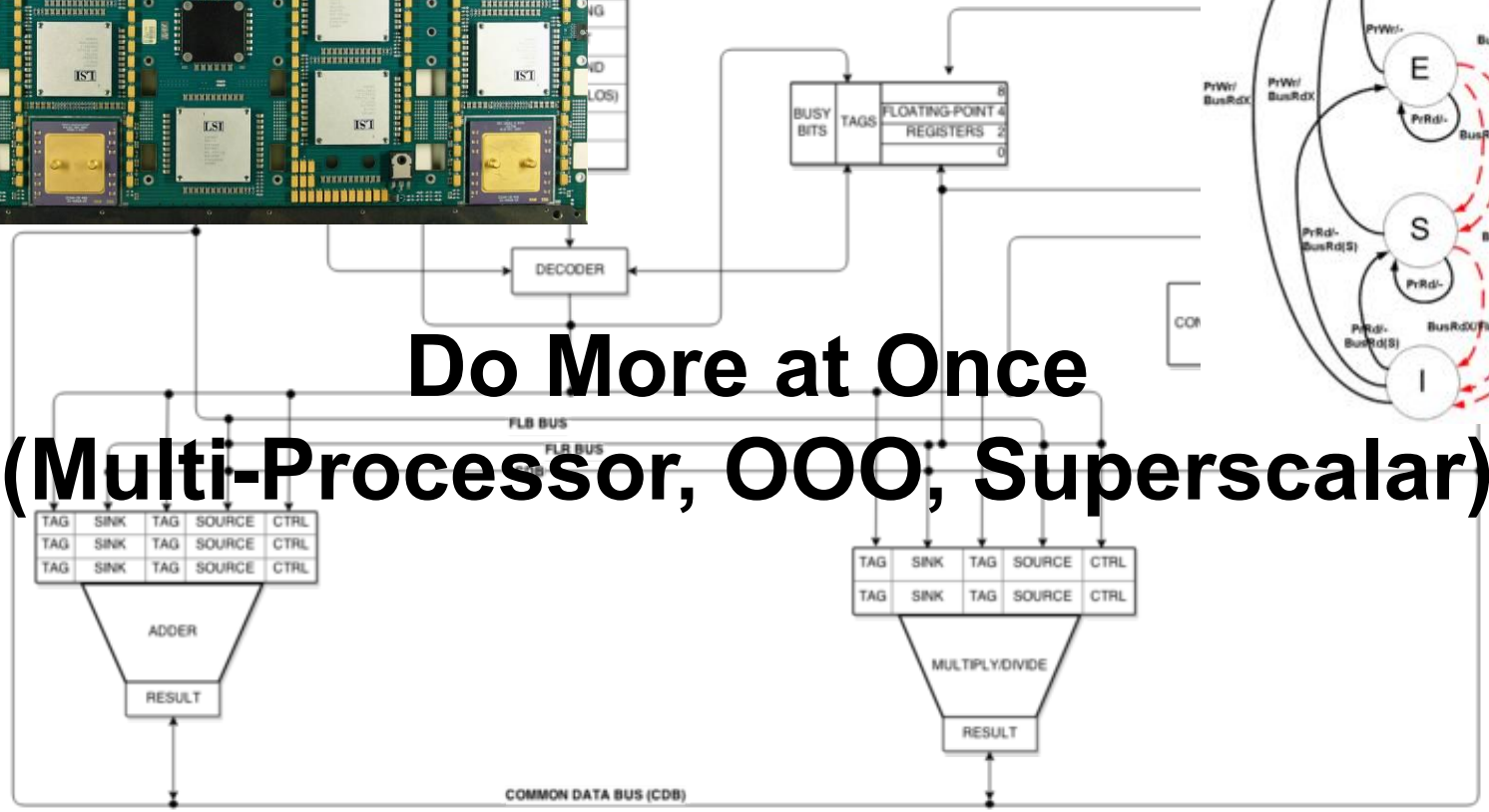
Associativity: Non-Associative, Set Associative, Fully Associative

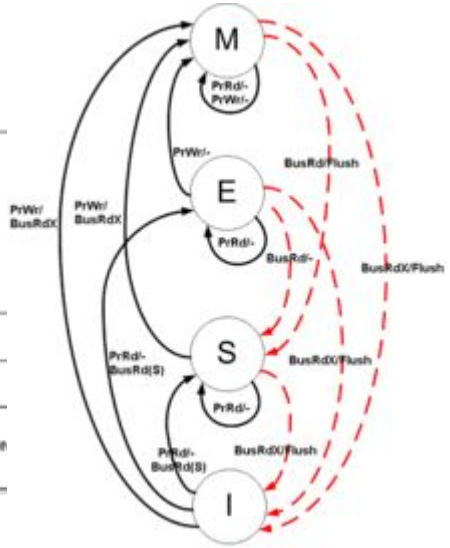


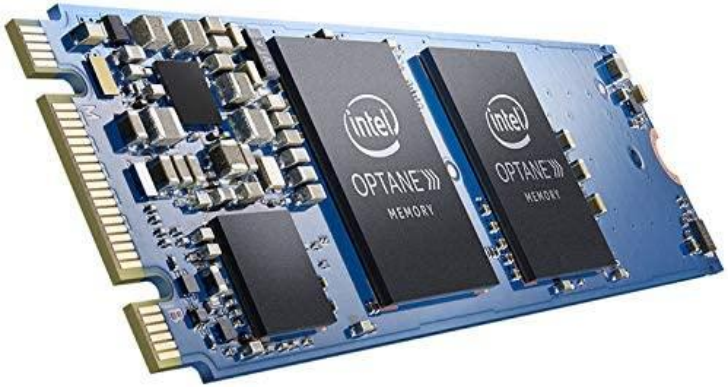
3. SEMESTER PREVIEW

Do More at Once
(Multi-Processor, OOO, Superscalar)

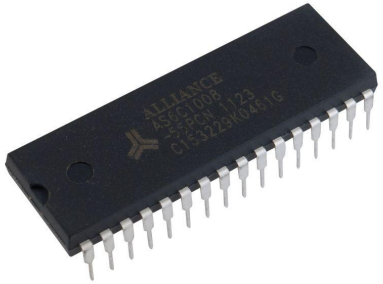


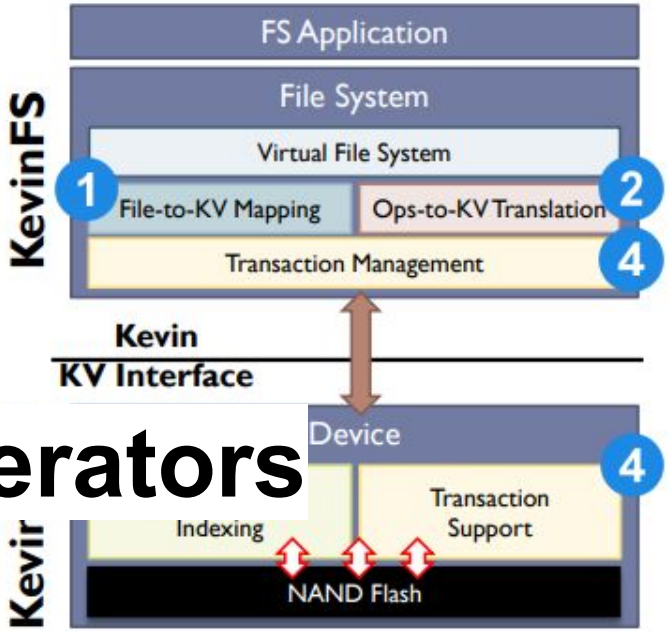




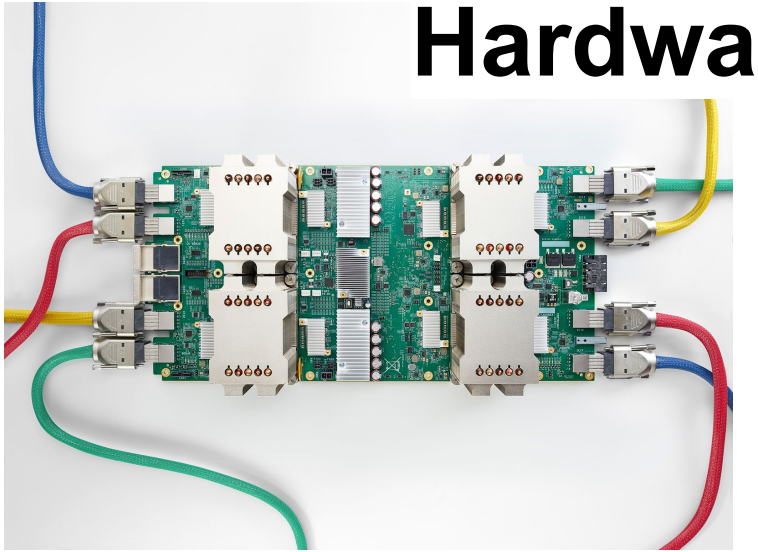


Memory and Storage

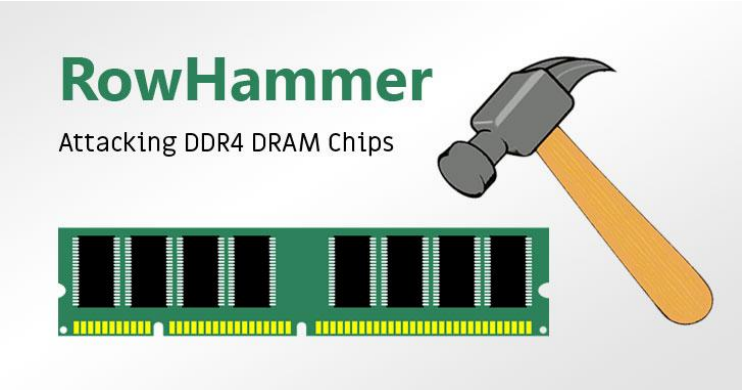




Hardware Accelerators



Hardware Security
(Spectre, Meltdown, RowHammer)



OSDI 22

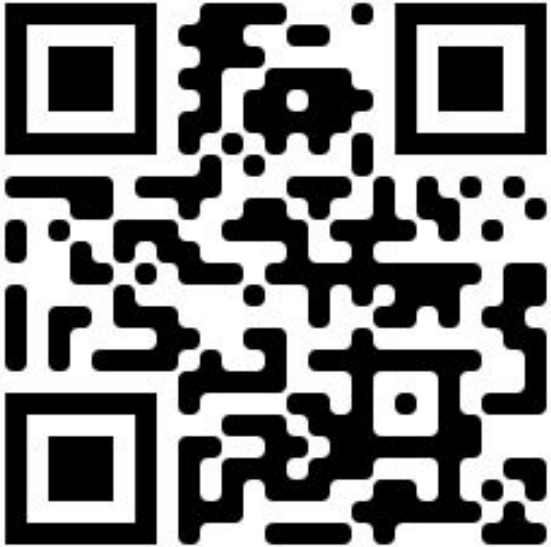
IEEE

Advancing Technology for Humanity

Paper Discussion

Have ideas? Message us!

usenix ASSOCIATION

ACM SIGARCH

Join Our Discord to Stay Updated